

Introduction

This document provides instructions for reproducing the PowerLLM benchmark results.

1. Prepare Datasets

The benchmark is evaluated on subsets of the original datasets. Each subset is sampled with a fixed random seed to ensure reproducibility.

A Python script, `benchmark/dataset_cut3.py`, is provided to sample data from the original datasets.

1.1 Download the Corpus and Datasets

The corpus and dataset JSON files are provided by LegalBench-RAG. You can download them from <https://www.dropbox.com/scl/fo/r7xfa5i3hdsbxex1w6amw/AID3890lvtm-ZLTkAPrw6k4?dl=0&rlkey=5n8zrbk4c08lbit3iixofmwg> (provided by the LegalBench-RAG authors).

LegalBench-RAG official Repository: <https://github.com/zeroentropy-ai/legalbenchrag>

Once downloaded, make sure the following folders are placed under this repository's `benchmark/` directory:

1. Move the `corpus` folder to `benchmark/corpus/`.
2. Move the dataset JSON folder to `benchmark/legalbench-datasets/`.

The reproduction commands in this document expect these files and folders to exist:

- `benchmark/corpus/cuad/...`
- `benchmark/corpus/maud/...`
- `benchmark/corpus/contractnli/...`
- `benchmark/corpus/privacy_qa/...`
- `benchmark/legalbench-datasets/cuad.json`
- `benchmark/legalbench-datasets/maud.json`
- `benchmark/legalbench-datasets/contractnli.json`
- `benchmark/legalbench-datasets/privacy_qa.json`

1.2 Prepare Mixed Dataset

Since LegalBench-RAG combines four datasets, we provide a Python script, `benchmark/build_mix_dataset.py`, to build paired mixed datasets for config tuning and validation.

The mixed split files referenced later in this document are generated artifacts, not repository source files:

- `benchmark/legalbench_mixed_config_q200_c20_seed42.json`
- `benchmark/legalbench_mixed_validation_q200_c20_seed42.json`

Generate these first from the original `benchmark/legalbench-datasets/*.json` inputs before running config tuning or the mixed-split benchmark commands.

Mixed Config + Validation Splits

Use the script to build paired mixed datasets for config tuning and validation that do not share source documents.

The script samples each input file twice with the same per-source quota:

- first for the `config` split
- then for the `validation` split from the remaining contexts

This keeps the command simple while ensuring the two outputs are disjoint by `context / file_path`.

The following command was used to build the datasets used in the benchmark.

```
venv/bin/python -m benchmark.build_mix_dataset \
  --input benchmark/legalbench-datasets/cuad.json \
    benchmark/legalbench-datasets/maud.json \
    benchmark/legalbench-datasets/contractnli.json \
    benchmark/legalbench-datasets/privacy_qa.json \
  --max-contexts 7 5 5 3 \
  --max-questions 70 50 50 30 \
  --min-per-context 10 \
  --seed 42 \
  --output-config benchmark/legalbench_mixed_config_q200_c20_seed42.json
  --output-validation benchmark/legalbench_mixed_validation_q200_c20_seec
```

Notes:

- `config` and `validation` use the same per-source quotas
- the two outputs do not share contexts
- each source must have enough eligible contexts to satisfy both splits
- `--max-contexts` and `--max-questions` are required for this dual-output flow
- the config split is the tuning dataset referenced again in `benchmark/retrieval_config_tuning_README.md`

1.3 Sample Questions from Each Full Dataset

For each dataset in LegalBench-RAG, the benchmark uses 200 questions, with a minimum of 10 questions per context.

```
# Sample 200 questions for MAUD dataset
venv/bin/python benchmark/dataset_cut3.py \
  --input-path benchmark/legalbench-datasets/maud.json \
  --exclude-dataset benchmark/legalbench_mixed_config_q200_c20_seed42.jsc
  --target-questions 200 \
  --target-contexts 20 \
  --min-per-context 10 \
  --seed 102 \
  --output no_overlap_dataset/legalbench_sample_maud_q200_c20_min10_seed1

# Sample 200 questions from ContractNLI dataset
venv/bin/python benchmark/dataset_cut3.py \
  --input-path benchmark/legalbench-datasets/contractnli.json \
  --exclude-dataset benchmark/legalbench_mixed_config_q200_c20_seed42.jsc
  --target-questions 200 \
  --target-contexts 20 \
  --min-per-context 10 \
  --seed 103 \
  --output no_overlap_dataset/legalbench_sample_contractnli_q200_c20_min1

# Sample 200 questions from CUAD dataset
venv/bin/python benchmark/dataset_cut3.py \
  --input-path benchmark/legalbench-datasets/cuad.json \
  --exclude-dataset benchmark/legalbench_mixed_config_q200_c20_seed42.jsc
  --target-questions 200 \
  --target-contexts 20 \
  --min-per-context 10 \
  --seed 101 \
  --output no_overlap_dataset/legalbench_sample_cuad_q200_c20_min10_seed1
```

```
# Sample 80 questions from PrivacyQA dataset
venv/bin/python benchmark/dataset_cut3.py \
  --input-path benchmark/legalbench-datasets/privacy_qa.json \
  --exclude-dataset benchmark/legalbench_mixed_config_q200_c20_seed42.jsc
  --target-questions 80 \
  --target-contexts 4 \
  --min-per-context 10 \
  --seed 104 \
  --output no_overlap_dataset/legalbench_sample_privacy_qa_q80_c4_min10_s
```

Note that the PrivacyQA dataset in LegalBench-RAG is a small variant with 194 questions in total. As a result, the command above uses the full eligible subset rather than a large re-sampled split.

These four sampled datasets were used for benchmark reporting and final validation.

2. Execute Config Tuning

To run the tuning or benchmark script, you need to set up `.env` first:

Add `.env` file in root folder, with following items:

```
# US Region LangSmith // Not required
LANGSMITH_TRACING=false // Disable by default
LANGSMITH_ENDPOINT=https://api.smith.langchain.com
LANGSMITH_API_KEY=...
LANGSMITH_PROJECT="###"

GOOGLE_API_KEY="..." // Required for Gemini API calls

CLOUDFLARE_ACCOUNT_ID="..." // Required for Qwen3-Embedding-0.6B-4bit-DWQ
CLOUDFLARE_AUTH_TOKEN="..."

OPENROUTER_API_KEY="..." // Required if using Qwen-3.5-9B-OpenRouter
```

Please note that results may vary and may not be exactly reproducible due to factors such as randomness.

For hyperparameter tuning, each trial was evaluated on a fixed 50-question subset of the 200-question tuning split. This subset was constructed by selecting 10 contexts and up to 5 questions per context to preserve cache reuse during chunking and retrieval experiments. The resulting subset contained 25 CUAD, 10 MAUD, 10 PrivacyQA, and 5 ContractNLI questions.

The following commands reproduce the tuning runs used in this work.

2.1 Budget-aware TPE search

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --search-method tpe \
  --study-name fixed_chunking_2000_tpe_q50_c10x5_20260425 \
  --n-trials 100 \
  --limit-contracts 10 \
  --limit-questions 5
```

This run fixes `chunk_size=2000`, `chunk_overlap=400`, and `rpf_k=60`, while tuning `similarity_k`, `bm25_k`, and `final_k` under the 22,000 -character context budget.

2.2 Budget-aware random search

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --search-method random \
  --study-name fixed_chunking_2000_random_q50_c10x5_20260425 \
  --n-trials 100 \
  --limit-contracts 10 \
  --limit-questions 5
```

This run uses the same search space and budget as the TPE run, but replaces adaptive search with random sampling.

2.3 Empirical grid baseline

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space empirical_grid_baseline \
  --search-method grid \
  --disable-budget-pruning \
  --study-name empirical_grid_baseline_no_budget_q50_c10x5_20260425 \
  --n-trials 108 \
  --limit-contracts 10 \
  --limit-questions 5
```

This run evaluates the manual grid baseline without budget pruning. The search space is:

- `chunk_size`: 512, 1000, 2000

- `similarity_k`: 4, 8, 12
- `bm25_k`: 40, 60, 80
- `final_k`: 8, 10, 12, 14

3. Execute Benchmark

After tuning, evaluate the selected configuration on the mixed validation split and on the four no-overlap per-dataset splits.

3.1 Optimal result with our tuning approach

The following commands reproduce the benchmark runs for the selected budget-aware random-search configuration:

```
# Mixed validation split
venv/bin/python -m benchmark.benchmark \
  --dataset benchmark/legalbench_mixed_validation_q200_c20_seed42.json \
  --retrieval-mode hybrid \
  --llm-mode concurrent \
  --chunking-strategy legal \
  --chunk-size 2000 \
  --chunk-overlap 400 \
  --similarity-k 15 \
  --bm25-k 73 \
  --final-k 11 \
  --rrf-k 60

# CUAD
venv/bin/python -m benchmark.benchmark \
  --dataset benchmark/no_overlap_dataset/legalbench_sample_cuad_q200_c20_
  --retrieval-mode hybrid \
  --llm-mode concurrent \
  --chunking-strategy legal \
  --chunk-size 2000 \
  --chunk-overlap 400 \
  --similarity-k 15 \
  --bm25-k 73 \
  --final-k 11 \
  --rrf-k 60

# PrivacyQA
venv/bin/python -m benchmark.benchmark \
```

```

--dataset benchmark/no_overlap_dataset/legalbench_sample_privacy_qa_q80
--retrieval-mode hybrid \
--llm-mode concurrent \
--chunking-strategy legal \
--chunk-size 2000 \
--chunk-overlap 400 \
--similarity-k 15 \
--bm25-k 73 \
--final-k 11 \
--rrf-k 60

# ContractNLI
venv/bin/python -m benchmark.benchmark \
  --dataset benchmark/no_overlap_dataset/legalbench_sample_contractnli_q2
  --retrieval-mode hybrid \
  --llm-mode concurrent \
  --chunking-strategy legal \
  --chunk-size 2000 \
  --chunk-overlap 400 \
  --similarity-k 15 \
  --bm25-k 73 \
  --final-k 11 \
  --rrf-k 60

# MAUD
venv/bin/python -m benchmark.benchmark \
  --dataset benchmark/no_overlap_dataset/legalbench_sample_maud_q200_c20_
  --retrieval-mode hybrid \
  --llm-mode concurrent \
  --chunking-strategy legal \
  --chunk-size 2000 \
  --chunk-overlap 400 \
  --similarity-k 15 \
  --bm25-k 73 \
  --final-k 11 \
  --rrf-k 60

```

3.2 Baseline

The following commands reproduce the benchmark runs for the strongest budget-compliant empirical grid baseline:

```
# Mixed validation split
venv/bin/python -m benchmark.benchmark \
  --dataset benchmark/legalbench_mixed_validation_q200_c20_seed42.json \
  --retrieval-mode hybrid \
  --llm-mode concurrent \
  --chunking-strategy legal \
  --chunk-size 2000 \
  --chunk-overlap 400 \
  --similarity-k 12 \
  --bm25-k 80 \
  --final-k 10 \
  --rrf-k 60

# CUAD
venv/bin/python -m benchmark.benchmark \
  --dataset benchmark/no_overlap_dataset/legalbench_sample_cuad_q200_c20_
  --retrieval-mode hybrid \
  --llm-mode concurrent \
  --chunking-strategy legal \
  --chunk-size 2000 \
  --chunk-overlap 400 \
  --similarity-k 12 \
  --bm25-k 80 \
  --final-k 10 \
  --rrf-k 60

# PrivacyQA
venv/bin/python -m benchmark.benchmark \
  --dataset benchmark/no_overlap_dataset/legalbench_sample_privacy_qa_q80
  --retrieval-mode hybrid \
  --llm-mode concurrent \
  --chunking-strategy legal \
  --chunk-size 2000 \
  --chunk-overlap 400 \
  --similarity-k 12 \
  --bm25-k 80 \
  --final-k 10 \
  --rrf-k 60

# ContractNLI
venv/bin/python -m benchmark.benchmark \
  --dataset benchmark/no_overlap_dataset/legalbench_sample_contractnli_q2
  --retrieval-mode hybrid \
  --llm-mode concurrent \
```



```
--chunking-strategy legal \  
--chunk-size 2000 \  
--chunk-overlap 400 \  
--similarity-k 12 \  
--bm25-k 80 \  
--final-k 10 \  
--rrf-k 60  
  
# MAUD  
venv/bin/python -m benchmark.benchmark \  
  --dataset benchmark/no_overlap_dataset/legalbench_sample_maud_q200_c20_  
  --retrieval-mode hybrid \  
  --llm-mode concurrent \  
  --chunking-strategy legal \  
  --chunk-size 2000 \  
  --chunk-overlap 400 \  
  --similarity-k 12 \  
  --bm25-k 80 \  
  --final-k 10 \  
  --rrf-k 60
```